CASSANDRAGARGOYLE · PORTUNIX ECOSYSTEM

VIM

The Layered Information Model: how a document learns to say what it means

CassandraGargoyle Community

Version 1.1.0 · Built 2026-10-25



Contents

PART 1 **Prologue** 3

1 Who Is the Customer? 4

PART 2 **How VIM Grew** 7

2 Sheets of Glass 8

3 From 1.0 to 2.1 11

PART 3 **Inside VIM** 15

4 Inside a vim.json 16

5 The Machine That Reads 22

6 A Promise in JSON Schema 26

PART 4 **Working with VIM** 30

7 Your First vim.json 31

8 Where VIM Is Going 36

PART 1

Prologue

One question about one invoice, asked twice: once of plain text and once of a layered model.

1 · Who Is the Customer?

CHAPTER 1

Who Is the Customer?

One Question, Asked Twice

On the last day of January 2026, someone in the team asked a language model a very simple question about an electricity bill: *who is the customer on this document?*

A person needs half a second for the answer. The name stands right there, under the word *Zákazník*, in the upper left corner of the page. Next to it, in a second column, there is the word *Adresát*, the addressee, with the same name and the same address under it. Any child who can read would point at the right block.

The model got the document as plain text, the way most tools still pass a PDF on: one line after another, from the top of the page to the bottom. It read for a while and then gave an honest answer. The customer, it said, *cannot be determined with certainty*. There was a candidate, a certain Ing. Jan Novák, but the relation to the label was not explicit. The first line of the text read *"Zákazník Adresát"*: two column headers had melted into one line. A hundred lines later, the name appeared twice on one line, and so did the address. The two columns of the page had been flattened into one stream of words. The confidence of the answer: **low**.

A few minutes later the same question went to the same model about the same bill. This time the document came as a `vim.json` file. The answer was short: Ing. Jan Novák, with his address, and the confidence was **high**. The model also explained why. In the file there was a block of type `labeled_content` with the header *Zákazník* and the name under it, and a separate block with the header *Adresát*. A layer above it held an entity of type `customer` that pointed back to the first block. Nothing had to be guessed.

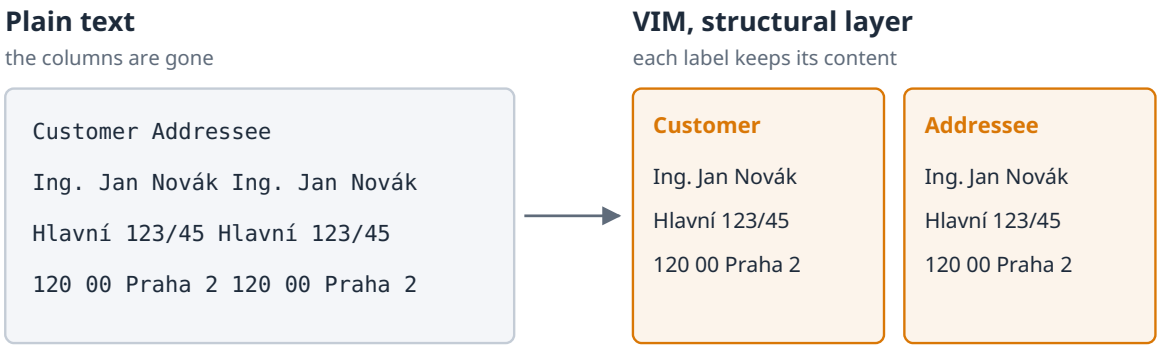


Figure 1.1 The same two-column header as plain text and as two labeled blocks of the structural layer

Both results are still kept in the repository of the parser, side by side, as the first experiment of a small research apparatus. They are not a benchmark. They are one question about one anonymized document. But they show in a few lines the whole reason for this book.

The Problem Is Not the PDF

It is easy to blame the format. PDF is everywhere, and it is famous for being hard to read by machines. But PDF was never meant to carry meaning. Adobe introduced it in 1993 to make a page look the same on every screen and every printer, and in 2008 it became the international standard ISO 32000. It does that job very well. A PDF knows where each glyph stands. It does not know that the glyphs under *Zákazník* form a name, or that the name belongs to the customer and not to the addressee.

The design notes of VIM say it plainly: *PDF is just a symptom*. The real problem is the **loss of semantics when data travels between systems**. A typical chain looks like this:

```
DB → reporting tool → PDF → human → manual transcription
```

At the start of the chain the data is perfect: a database knows exactly which value is the customer, which is the amount and which is the currency. Then a reporting tool prints it on a page, the page travels to someone else, and on the other side a person, or today a program, has to rebuild the meaning from the picture. What the receiver wants is not the PDF. The receiver wants values, relations, meaning, and proof that the values really come from this document.

The Answer Is Not One Format

When we look at this problem for the first time, we want one perfect format that holds everything. The history of documents says it does not exist. Libraries met the same question when they began to digitize books: in June 2004 the EU project METAe released the first version of ALTO, an XML format that records where each word, line and column stands on a scanned page. ALTO describes the layout very well, and the Library of Congress maintains it to this day. But it says nothing about what a word means. Other formats describe meaning very well and know nothing about the page.

The design notes of VIM draw the conclusion in two sentences that open the whole concept:

> There is no single universal format for all files. There is a universal layered model for information representation.

The question is therefore not *which format stores everything*. The question is *how to store the result of understanding a file*. The answer of VIM is a stack of layers. The lowest layer says what is physically on the page. The next one says how it is grouped. The third one says what it means. Every value on every layer can point back to the file, the page and the rectangle it came from, and it says how sure the machine was.

The Name

VIM is short for **Vrstvený informační model**, Czech for *Layered Information Model*. The model was born in a Czech team, on Czech invoices and Czech payment QR codes, and the name kept its origin. In English the letters do not fit the words, so the book uses VIM as a proper name, the way

people say PHP or GNU without thinking of what the letters once meant. For a while some documents of the ecosystem expanded VIM as *Versatile Information Model*. That expansion kept the letters and lost the point: the important word is *layered*.

The name has one more small trap. A reader who works with text editors will think of the editor Vim at once. The two have nothing in common except three letters. The VIM of this book is a data model; its files end with `.vim.json`, and nobody has to learn how to exit it.

What This Book Is About

This book tells the story of VIM: how a parser for invoices, written in a few intense days at the end of January 2026, grew within nine months into the shared result container of a whole ecosystem of document tools. It shows what a `vim.json` contains, layer by layer, on a real file produced by the real parser. It looks inside the machine that writes it, and at the contract in this repository that tells every producer and every reader what they may expect. And it shows how to make your first `vim.json` and how to check it.

The next chapter starts with the idea itself: what it means to read a document in layers.

Sources

- Research results of the experiment: `research/results/` of the `vim-parser` plugin in the `portunix-plugins` repository (31 January 2026)
- Design notes *Universal "Perfect" Format for Storing Information from Any File*, `plugins/vim-parser/README.md` in `portunix-plugins`
- PDF: introduced by Adobe in 1993, ISO 32000-1 in 2008
- ALTO (XML): version 1.0 by the METAe project in June 2004, maintained by the Library of Congress since 2009

PART 2

How VIM Grew

The idea behind the layers, and nine months in which a parser for invoices became a shared model of the whole ecosystem.

2 · Sheets of Glass

3 · From 1.0 to 2.1

CHAPTER 2

Sheets of Glass

Reading Is Not One Act

Watch a person who reads an invoice for the first time. The eyes do not run from the first letter to the last. They first take in the page as a picture: a bold line at the top, two blocks of text side by side, a table, a black square in a corner. Then they group: this block is the supplier, that one is the customer, this grid is the list of items. Only then comes meaning: the number at the bottom right is the total, in Czech crowns, and the black square is a QR code with which the bill can be paid.

Reading is not one act. It is at least three, and each one stands on the one below it. If we want a machine to keep the result of reading, it is natural to keep all three, and to keep them apart. Imagine three sheets of glass laid over the page. On the lowest one, you trace what is really there. On the second one, you draw frames around what belongs together. On the top one, you write what it means. Each sheet can be lifted and looked at alone, and each mark on a higher sheet stands exactly over the marks it was made from.

This picture is VIM.

Five Principles

The design notes of VIM name five principles. They are worth reading slowly, because every later decision in the format goes back to one of them.

Separate content, structure and meaning. The text `123 456` is content. The fact that it sits in a cell, in a column, in a table, is structure. The fact that it is a net amount in Czech crowns without VAT is meaning. Three different things, stored in three different places.

Be layered. One layer is never enough. The notes put it as a short chain: raw, layout, structure, semantics. A model that keeps only the last step throws away the evidence; a model that keeps only the first step leaves all the work to the reader.

Preserve provenance. Every value must be able to say where it came from: which file, which page, which region of the page. Without that, a value is a rumour.

Be bidirectional. From the model, a program must be able to compute: sum the totals of a year of invoices, for example. And from the model, the document must be at least roughly reconstructable. A later chapter shows a page of an invoice redrawn from nothing but its `vim.json`.

Handle uncertainty. No machine reads a document with full certainty. So every result carries a confidence, the method that produced it and, where it makes sense, the alternatives that were considered and rejected.

The Three Questions

The first three layers of VIM answer three questions, and their names are short enough to remember:

Layer	Question	What it holds
`physical`	What is there?	Text with exact positions and fonts, barcodes, tables, graphics, the full text
`structural`	What does it look like?	Blocks with a label and a body, their type and importance, barcodes with the blocks near them
`semantic`	What does it mean?	Entities such as a customer or a supplier, payments read from QR codes, word stems for search

The order is the depth of interpretation. The physical layer interprets almost nothing: it says that the string *Odběratel* stands at a certain position in bold. The structural layer interprets a little: it says that this bold string is the label of a block with three lines under it. The semantic layer interprets the most: it says that the block describes the customer, and it says so with a confidence of 0.8, not 1.0.

Each higher layer refers to the lower one by a stable identifier instead of copying it. A structural block `block_4` has the same ID as the physical block it was made from. A semantic entity says `source_block: block_4`. That is the mark on the upper sheet of glass standing exactly over the mark below.

What Happened to the Fourth Layer

The original design notes had a fourth layer: *Provenance and trust*, with the question *why do we believe it?* It was meant to hold the source, the page, the rectangle, the confidence and the method of every value.

When the format was built, this layer did not become a separate sheet. It became something better: a property of every element on every sheet. A text element in the physical layer carries its `bbox`, its `page` and its `confidence`. A structural barcode carries the IDs of the blocks near it and a confidence. A semantic entity carries the ID of its source block and a confidence. Provenance is not a place you go to; it travels with each value. A consumer never has to join two layers to learn where a number came from.

Layers That Came Later

The three interpretation levels are a closed set. A fourth level of interpretation would change the core idea of VIM, and the architecture decisions of the ecosystem say so explicitly. But a document can be seen not only deeper, but also from another side. Over time, VIM got three more layers of a different kind, the **domain layers**:

Layer	Since	Question	Producer
`toc`	1.4	How do I find things in it?	vim-parser
`extraction`	2.0	What does this form hold?	form recognition (reco) and others
`prescriptive`	2.1	What does this document require?	an LLM-based producer, checked by a person

A domain layer does not interpret the page more deeply. It holds a view built for one purpose: navigation in a long book, the values of a filled form, the rules written in a specification. Every domain layer refers to the interpretation levels by ID, so the anchors stay in one place.

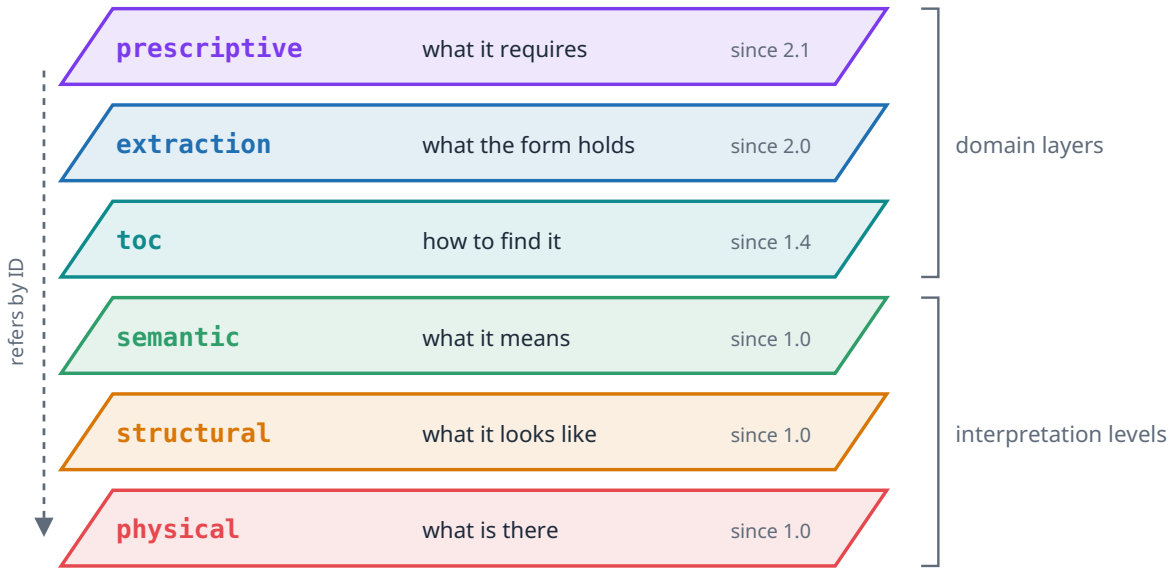


Figure 2.1 The six layers of VIM: three interpretation levels below, three domain layers above, which refer down by ID

All layers are optional. A document from an image scanner may have only the `extraction` layer. A long PDF book may have `physical`, `structural` and `toc`. A reader ignores the layers it does not know. This is what lets very different tools share one container without stepping on each other.

A Model, Not a File Format

There is one more idea in the design notes that is easy to miss. The *perfect format*, the notes say, is not one thing but a **pipeline plus a data model**. VIM is the model. It fixes what the result of understanding a document looks like. It does not fix how the understanding happens. One tool may read text from the PDF directly; another may run OCR on a scan; a third may ask a language model. They all write the same layers, and every value says which method produced it.

This is also why VIM could grow so fast. Its first version was made for invoices in PDF. Within months it held the results of form recognition from scanned images and the requirements of technical specifications. The model did not have to change its idea; it only had to gain new sheets of glass. How that happened, and in what order, is the story of the next chapter.

Sources

- Design notes *Universal "Perfect" Format for Storing Information from Any File*, `plugins/vim-parser/README.md` in `portunix-plugins`
- VIM format specification v1.5 and v2.0, `docs/architecture/` in `portunix-architecture`
- ADR-009 *Criteria for Adding a New VIM Layer* (9 October 2026), `portunix-architecture`

CHAPTER 3

From 1.0 to 2.1

Two Days in January

Most formats are born slowly, in committees, over drafts and comments. VIM was born in a rush. On Friday, 30 January 2026, a new plugin appeared in the repository of the Portunix plugins. Its first commit was called *Initial plugin scaffold with VIM architecture*. By the evening of the same day, the plugin could cut the text of a PDF into blocks with exact positions, show the result as an HTML page, and find two-dimensional barcodes on the page. The format had its first two versions: **1.0** with the physical and structural layers, and **1.1** with barcodes.

The next day, Saturday, was even busier. The plugin was split into modules and got a system of plugins of its own, built on the Java library PF4J, so that each kind of extraction could live in its own package and be added or removed without rebuilding the core. An orchestrator learned to run the plugins in the right order by itself. Tables arrived, then document metadata and the full text (**1.2**), then images, and in the evening a lemmatizer that reduces words to their stems for search (**1.3**). On the same day, the experiment from the prologue was run: the question *who is the customer* asked of plain text and of a `vim.json`. The layered model passed its first test while it was barely one day old.

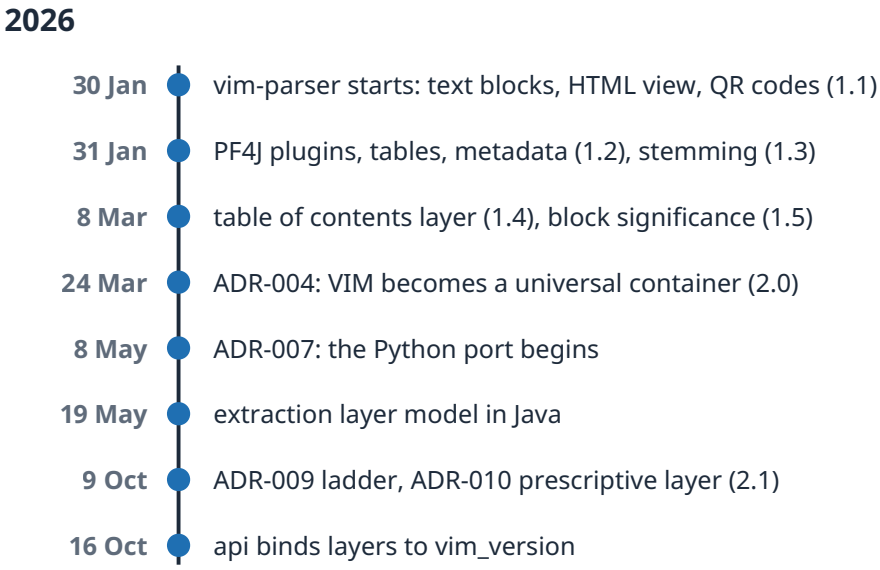


Figure 3.1 The year 2026 of VIM: from the first commit of the parser to the contract of version 2.1

The Long Tail of Real Documents

After the first rush came the slow work that every document tool knows: real documents. In early February the lemmatizer got native backends for Czech, written in C++ and connected through JNI, because Czech is a language with rich word forms and a simple stemmer does not do it justice. Table detection moved into its own universal plugin, so that other tools could use it too.

At the end of February, an article about artificial intelligence exposed a typographic detail: a **drop cap**, the large first letter of a paragraph. To a machine that groups characters by position, a drop cap is a lonely giant letter that belongs to no line. The parser learned to detect it and merge it back into the word it starts, and the text element got the fields `drop_cap_merged` and `drop_cap_bbox`. This is how VIM usually grows: not by grand plans, but by one stubborn document at a time.

In March, the parser met books. A script downloaded sample books, among them the *Meditations* of Marcus Aurelius, and the team taught the parser to read a table of contents, both the explicit one printed in the book and one inferred from the headings. On 8 March this became the `toc` layer and version **1.4**. On the same day, blocks got a **significance**, primary or auxiliary, so that a page number or a running header no longer looks as important as the text, and the default version became **1.5**.

A small quirk from this time is still visible today. In the 1.x line, the parser sets `vim_version` by the features a document actually uses. A document with a table of contents says `"1.4"`, even though the parser knows version 1.5. In the 1.x years, `vim_version` was closer to a list of features than to a version of a contract. Version 2 changed that.

A Container for Everyone

On 24 March 2026, an architecture decision changed the role of VIM. Its title was short: *VIM as Universal Result Container* (ADR-004).

The cause was a new producer. The Portunix ecosystem was getting a framework for form recognition, `portunix-reco`, written in Python. It reads scanned images, not PDFs. One logical document may come from many images. It extracts fields from predefined zones of a form, and a table may run over several pages and has to be merged into one. VIM 1.5 could not hold any of that: its physical layer assumed PDF text with fonts, and its semantic layer knew a fixed set of entity types.

The decision kept everything that existed and added what was missing. The four existing layers stayed unchanged; a 1.5 document stayed a valid 2.0 document. A new layer, `extraction`, received the results of form processing: a reference to the template that drove the extraction, the raw results of each source with full provenance, and the merged logical document. Every layer became optional. The parameters of every extraction step were kept, so that a debugging tool can take a `vim.json`, find the source image and run the step again with identical settings. And every change of a value after extraction, by an operator, a rule or a language model, is recorded as a **revision**, while the original raw value never changes.

The decision ended with a sentence that matters for this repository: the schemas of the new format go to `api/contract/schemas/`, as language-independent JSON Schema contracts.

Two Languages, One Model

By May, the parser had about 20,000 lines of Java in thirteen Maven modules, and the Python world of reco needed the same logic natively. Calling Java from every Python deployment meant a JVM next to every worker and a start-up cost for every document. Rewriting 20,000 lines by hand was not realistic for one architect.

ADR-007, accepted on 8 May 2026, chose a third way: a translation tool. A new module of the Dream project, `dream-java2py`, reads the Java source with a real parser of the Java language, maps the libraries (PDFBox, ZXing, Lucene) to their Python counterparts, and lets a language model write idiomatic Python, file by file, with a person reviewing every file before it lands in the sub-package `ptx_reco.vim`. The port grows incrementally, and the tool keeps a record of what is translated and what is still waiting.

The same month brought two more things. On 15 May, a separate **research lab** for the parser was founded, where extraction problems are collected as long-lived catalog entries and measured in versioned challenges. And on 19 May, the Java model of the `extraction` layer was written; the next day the parser was released as version 0.1.8.

A Ladder and a New Layer

New kinds of information kept arriving, and the team noticed that each one had been decided alone: barcodes became a collection in the physical layer, the table of contents became a layer of its own. On 9 October 2026, ADR-009 turned these habits into a rule: a **decision ladder**. New information first tries to be an optional field on an existing element, then a collection in an existing layer, and only as a last step a new layer, with three gates and four criteria to pass. A later chapter walks through the ladder in detail.

On the same day, the first candidate passed it. ADR-010 added the `prescriptive` layer and version **2.1**. It holds what a document *prescribes*: the requirements, recommendations and permissions of a specification, the references to other documents and the terms it defines. A language model proposes the provisions, a person confirms or rejects them, and every provision points to the page and the block it came from.

The same day, the schemas of the new layer arrived in this repository. Over the next two days, the Java parser and the Python port both got the model of the prescriptive layer, and the Java model of the extraction layer was aligned with the contract, defect by defect. On 16 October, the contract made the link between layers and versions strict: a document with the `prescriptive` layer must say `"2.1"`, a document with the `extraction` layer must say `"2.0"` or `"2.1"`. The `vim_version` of 2.x is a promise, no longer a list of features.

Versions at a Glance

Version	What it added	First seen
1.0	Text blocks: physical and structural layers	30 January 2026
1.1	Barcodes	30 January 2026
1.2	Document metadata and full text	31 January 2026

Version	What it added	First seen
1.3	Stemming	31 January 2026
1.4	Table of contents layer	8 March 2026
1.5	Graphics, tables and graphic elements; the default version of the parser	8 March 2026
2.0	`extraction` layer, all layers optional, sources, revisions	specified 24 March 2026
2.1	`prescriptive` layer, `property` on revisions	specified 9 October 2026

Nine months separate the first commit from version 2.1. In that time VIM went from a plugin that reads invoices to a model shared by a Java parser, a Python port, a form recognition framework, an invoice extraction pipeline and the contract in this repository. The next chapter opens a real `vim.json` and looks at what is inside.

Sources

- Commit history of `plugins/vim-parser` in `portunix-plugins` (30 January to 11 October 2026)
- ADR-004 *VIM as Universal Result Container* (24 March 2026), ADR-007 *Port vim-parser to Python via Dream Translation Tool* (8 May 2026), ADR-009 *Criteria for Adding a New VIM Layer* and ADR-010 *VIM Prescriptive Layer* (9 October 2026), `portunix-architecture`
- CHANGELOG.md of this repository: VIM prescriptive layer (#42) and the binding of layers to `vim_version` (#51)
- Commit history of `src/ptx_reco/vim` in `portunix-reco` and of `portunix-vim-parser-research-lab`

PART 3

Inside VIM

What a vim.json holds, the machine that writes it, and the contract that keeps every producer and reader honest.

4 · Inside a vim.json

5 · The Machine That Reads

6 · A Promise in JSON Schema

CHAPTER 4

Inside a vim.json

A Small Invoice

To look inside VIM, we need a document. For this book we made one: a one-page invoice from an invented glass company, *Sklo a vrstvy s.r.o.* (Glass and Layers Ltd.), to an invented customer in Brno. It has a title, two address blocks side by side, an issue date, a small table with two items, a total and a QR code for payment in the Czech SPAYD format. Every name and number in it is made up, so it can be printed in a public book.

We gave the PDF to the real parser, version 0.1.8, with its default settings:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto -o faktura-2026-0042.pdf.vim.json faktura-2026
-0042.pdf
```

The result is a JSON file of about 31 kilobytes. Everything in this chapter is copied from it; only long arrays are shortened. The PDF, the `vim.json` and the script that made the invoice are stored next to the book, so you can repeat the run.

The Envelope

Every VIM document has the same three parts at the top:

```
{
  "vim_version" : "1.4",
  "document" : {
    "source" : "faktura-2026-0042.pdf",
    "pages" : 1,
    "metadata" : {
      "encodingFormat" : "application/pdf",
      "contentSize" : 11122,
      "inLanguage" : "cs",
      "producer" : "WeasyPrint 70.0"
    }
  },
  "layers" : { "physical" : { }, "structural" : { }, "semantic" : { }, "toc" : { } }
}
```

`vim_version` says `"1.4"` because the document has a table of contents layer; the previous chapter explained this habit of the 1.x line. `document` describes the source: its file name, its page count and metadata with names taken from the Schema.org vocabulary (`encodingFormat`, `inLanguage`). The parser detected the language as Czech by itself. And `layers` holds the sheets of glass.

The Physical Layer: What Is There

The physical layer of our invoice has five text blocks, one barcode, one table, one graphic element and the full text. A block is a group of text elements that stand close together. Here is the start of the block with the customer:

```
{
  "id" : "block_4",
  "bbox" : { "x" : 299.1, "y" : 155.7, "width" : 84.4, "height" : 44.8 },
  "page" : 1,
  "text_elements" : [ {
    "text" : "Odběratel",
    "font_size" : 14.0,
    "font_name" : "UWGIWA+DejaVu-Sans-Bold",
    "confidence" : 1.0
  }, {
    "text" : "Jana Příkladová",
    "font_name" : "XKQQSR+DejaVu-Sans"
  } ]
}
```


Every element knows its rectangle on the page (`bbox`, in PDF points), its page and its font. The font name ends with *Bold* for the label and not for the name; that small detail is what lets the parser find labels later. The confidence is 1.0, because text taken directly from a PDF is not guessed. A text element read by OCR from a scan would carry a lower number.

The barcode is kept exactly as it was decoded, with its position:


```
{
  "id" : "barcode_1",
  "page" : 1,
  "format" : "QR_CODE",
  "raw_content" : "SPD*1.0*ACC:CZ6508000000192000145399*AM:1250.00*CC:CZK*X-
VS:20260042*MSG:Faktura 2026-0042"
}
```


This is the principle of bidirectionality at work. The physical layer holds enough to draw the page again. The next figure was drawn by a script from nothing but the `vim.json`: every word stands where the parser saw it, every red frame is a physical block, and the green boxes on the right show what the semantic layer made of them.

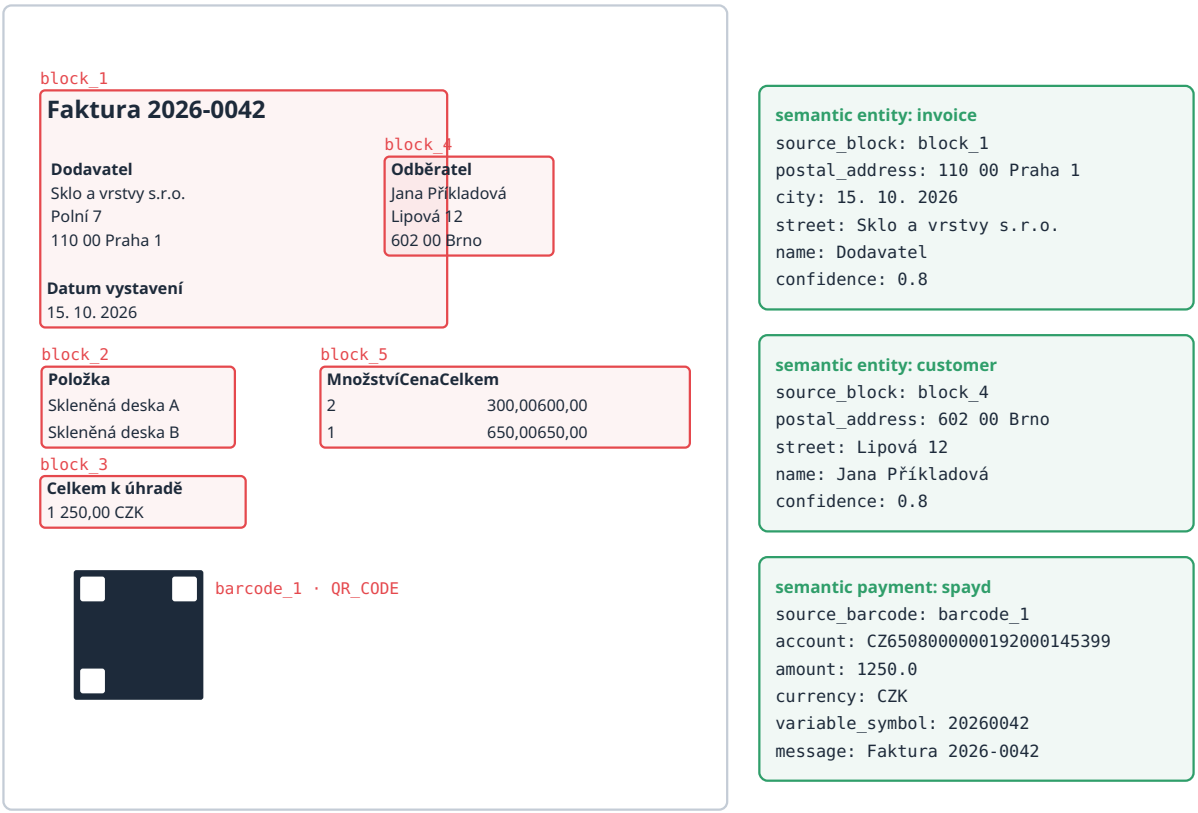


Figure 4.1 The sample invoice redrawn from its physical layer, with the blocks and the semantic results

The Structural Layer: What It Looks Like

The structural layer gives each physical block a shape. It keeps the same IDs:

```
{
  "id" : "block_4",
  "type" : "labeled_content",
  "significance" : "primary",
  "page" : 1,
  "header" : "Odběratel",
  "content" : [ "Jana Příkladová", "Lipová 12", "602 00 Brno" ]
}
```

The bold first line became the `header`, the rest became `content`. This is exactly the structure that the language model in the prologue was missing in plain text. The barcode also gets a structural record: its content type (`spayd`) and the list of blocks near it, so that a reader can find the text that explains a code.

The Semantic Layer: What It Means

On the semantic layer, the parser turns blocks into entities and codes into payments:

```
"entities" : [ {
  "type" : "customer",
  "source_block" : "block_4",
  "fields" : { "name" : "Jana Příkladová", "street" : "Lipová 12", "postal_address" : "602
00 Brno" },
  "confidence" : 0.8
} ],
"payments" : [ {
  "type" : "payment",
  "source_barcode" : "barcode_1",
  "format" : "spayd",
  "fields" : { "account" : "CZ6508000000192000145399", "amount" : 1250.0,
               "currency" : "CZK", "variable_symbol" : "20260042", "message" : "Faktura
2026-0042" },
  "confidence" : 1.0
} ]
```


Two things are worth noticing. First, every semantic result points to its source: `source_block` or `source_barcode`. Second, the confidences differ. The customer was found by matching the label *Odběratel* against known headers, which is a heuristic, so it gets 0.8. The payment was decoded from a standard format with all required fields present, so it gets 1.0.

What the Parser Got Wrong

An honest book has to show the other half too. The same file contains three mistakes, and each of them teaches something.

The title swallowed the supplier. Block `block_1` contains the title *Faktura 2026-0042*, the supplier's address and the issue date, because they stand in one column close to each other. The structural layer therefore took the title as the header of the whole block, and the semantic layer produced an entity of type `invoice` whose `name` is *Dodavatel* (supplier) and whose `city` is *15. 10. 2026*. The confidence is still 0.8: the heuristic does not know that it was fooled.

The table columns melted. In the physical block of the table, the cells *300,00* and *600,00* became one string, `300,00600,00`, because the parser glued characters on one line without a space between columns. The table detector, working separately, found a grid of twelve rows and four columns over a large part of the page, with cells like *Faktura 202* and *6-004*. The full text from the metadata extractor, on the other hand, has the table rows right: *Skleněná deska A 2 300,00 600,00*.

The table of contents found headings in an invoice. The `toc` layer lists the title and three block headers as entries, with the source `heuristic`. For an invoice this is noise; for a book it would be useful.

None of this is hidden. Every wrong value carries the ID of the block it came from, the method and a confidence, and a consumer can check it. Multi-column layout and tables are exactly the problems that the research lab of the parser keeps as catalog entries, as the last chapter shows.

The Extraction Layer: What the Form Holds

Our invoice has no extraction layer; the parser does not produce it. The contract in this repository has a small example of it, `form-extraction.vim.json`: one scanned form, one field. It shows the idea of revisions better than any description:

```
{
  "field_id" : "order_number",
  "label" : "Order number",
  "value" : "A-1007",
  "raw_value" : "A-I007",
  "data_type" : "string",
  "confidence" : 0.71,
  "revisions" : [ {
    "revision" : 1,
    "old_value" : "A-I007",
    "new_value" : "A-1007",
    "actor" : "rule",
    "actor_id" : "rule:digit_lookalikes",
    "reason" : "character_replacement"
  } ]
}
```

The OCR engine read a capital *I* where the form has the digit *1*. A rule that knows lookalike characters corrected it. The `raw_value` keeps what the engine really read and never changes; the `value` shows the current state; the revision says who changed it and why. If an operator later corrects the value again, a second revision is added.

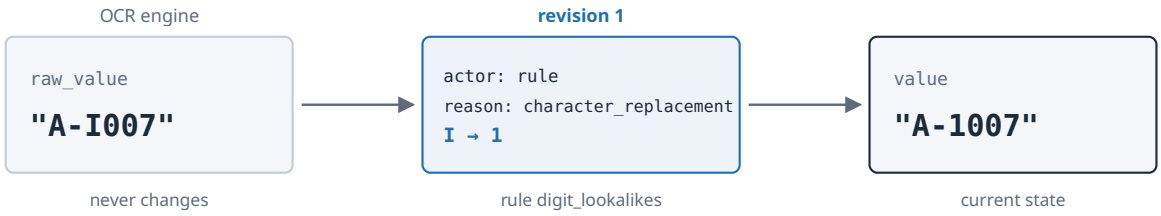


Figure 4.2 One field of the extraction layer: the raw value of the engine, a revision by a rule, and the current value

Around the fields, the layer holds a reference to the template that defined the zones (`template_ref`), one entry per source with its provenance (`sources`), and the merged logical document (`document`), where fields from many pages and tables split over pages come together.

The Prescriptive Layer: What It Requires

The second example of the contract, `definition-file.vim.json`, shows the youngest layer. Its source is a twelve-page document with rules for the title blocks of technical drawings. One of its provisions:

```
{
  "provision_id" : "p-5.3.2-1",
  "kind" : "requirement",
  "clause" : "5.3.2",
  "text" : "The title block shall contain the drawing number.",
  "anchors" : [ { "page" : 7, "element_type" : "block", "element_id" : "block_41" } ],
  "relations" : [ { "provision_id" : "p-5.3.1-1", "relation" : "refines" } ],
  "verification" : { "method" : "automatic", "subject" : "drawing",
                     "rule" : { "field" : "drawing_number", "check" : "present" } },
  "status" : "confirmed",
  "confidence" : 0.82
}
```

A provision is a requirement, a recommendation or a permission. It has a stable ID, the number of its clause, the verbatim text, and an anchor into the physical layer of the same document: page 7, block 41. It can refine another provision, depend on it, be an exception to it or conflict with it. It says how it can be checked: automatically, by a test or by a person. And it has a status. A language model proposes provisions as `proposed`; only a person makes them `confirmed`. In the example, the revision history shows that a reviewer changed the kind of this provision from *recommendation* to *requirement*, with the comment *The verbal form is shall.*

Six Layers in One Table

Layer	Main content	Typical producer
`physical`	`blocks` with text elements, `barcodes`, `tables`, `graphics`, `fulltext`	vim-parser
`structural`	`blocks` with header, content, type and significance; `barcodes` with nearby blocks	vim-parser
`semantic`	`entities`, `payments`, `stemming`	vim-parser
`toc`	`entries` with title, page, level, block and source	vim-parser
`extraction`	`template_ref`, `sources`, `document`; fields with `revisions`	reco and others
`prescriptive`	`definition`, `provisions`, `references`, `terms`	LLM-based producer, reviewed by a person

The next chapter looks at the machine that writes the first four layers: how a PDF travels through the plugins of the parser and comes out as a `vim.json`.

CHAPTER 5

The Machine That Reads

A Workshop, Not a Single Machine

A good picture of `vim-parser` is not one big machine with one input and one output. It is a workshop with specialists. One of them reads the metadata of the file. Another one cuts the text into blocks. Another one looks for barcodes, another one for tables, another one for pictures. When the barcode specialist is done, the payment specialist reads the codes it found. At the end, someone puts the table of contents together from the headings. Each specialist knows one job and leaves its results on a shared table for the others.

In the code, the specialists are **plugins** and the shared table is the **plugin context**. The workshop manager is an **orchestrator** that decides who works when.

Plugins in Packages

The parser is a Java 21 project of more than a dozen Maven modules. The core is a single "fat" JAR with the command line, the orchestrator and the plugin runtime. Every plugin is a separate ZIP file in a `plugins/` directory next to it. The plugin runtime is PF4J, a small plugin framework for Java: it finds the ZIP files at start, loads each one with its own class loader and asks it what it can do.

This design came on the second day of the parser's life, and it was not an accident. A plugin can be added, replaced or removed without rebuilding the core. The native lemmatizer backends for Czech, written in C++, live in their own plugins; a deployment that does not need them simply does not have the ZIP files.

Plugins come in kinds, defined by three extension points of the API module:

Kind	Extension point	Job	Plugins
Extractor	`ExtractorPlugin`	Read raw data from the file	metadata, fulltext, barcode, graphics, table, lemmatizer
Parser	`ParserPlugin`	Interpret data that extractors found	spayd
Visualizer	`VisualizerPlugin`	Make a view of a finished `vim.json`	html

The table of contents plugin is an extractor too, but it runs in its own phase, *transform*, because it builds on the text that other plugins found.

Who Works When

Each plugin declares which plugins it needs before it can start. The barcode plugin needs nothing. The SPAYD plugin needs the barcodes. The table of contents needs the full text. The orchestrator reads these declarations, builds a dependency graph and turns it into an execution plan, phase by phase. You can ask for the plan without running it. For the sample invoice of the previous chapter, the parser answers:

```
VIM Parser - Execution Plan
===========================
Input: faktura-2026-0042.pdf (10,9 KB, 1 page)

Discovered plugins: 8
Auto-enabled: 7

Execution sequence:
  Phase 1: EXTRACT
    [1] metadata     (no dependencies)
    [2] fulltext     (no dependencies)
    [3] barcode      (no dependencies)
    [4] table        (no dependencies)
    [5] graphics     (requires: barcode)

  Phase 2: PARSE
    [6] spayd        (requires: barcode)

  Phase 3: TRANSFORM
    [7] toc          (requires: fulltext)

Excluded from auto mode:
    [-] lemmatizer   (autoEnabled=false)
```

This is the output of `auto --dry-run` of version 0.1.8, copied without changes. The lemmatizer is installed but not enabled by default; it runs only when you ask for stemming with `--stemming`. That is the meaning of the word *auto* in the command: with zero configuration, the orchestrator runs every plugin that is enabled by default, in an order that respects every dependency.

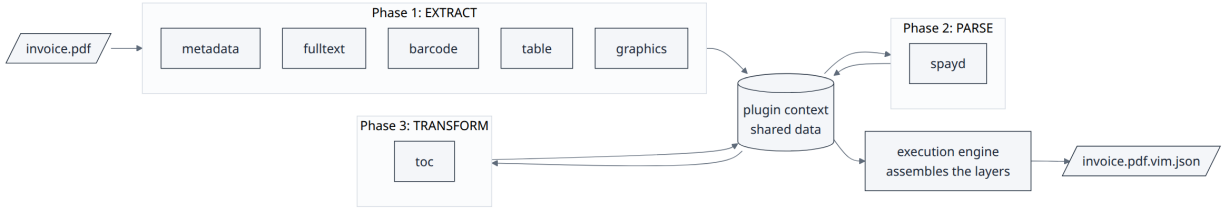


Figure 5.1 The way of a PDF through the parser: extractors, parsers and the transform phase leave their results in the shared context, and the engine assembles the layers

The Shared Table

Plugins never call each other. They talk through the shared data of the plugin context, a map from names to values. The text block extractor puts its blocks under `textBlocks`, the barcode extractor puts its codes under `barcodes`, the SPAYD parser reads `barcodes` and puts `spaydPayments` back. By default the plugins run one after another; the configuration file `vim-parser.json` can switch the execution mode to `thread`, and then extractors that do not depend on each other run in parallel. Either way, the shared map is the only place where they meet.

When all phases are done, the **execution engine** takes the values from the map and assembles the `VimDocument`: the physical blocks go to the physical layer, a structural block is derived from each physical block, the payments go to the semantic layer, the table of contents to its own layer. Two steps happen only here, after the plugins: the semantic entities are found by matching block headers against known labels, and every barcode gets the list of text blocks near it. This is also where `vim_version` is set by the features the document uses, the habit described in the second chapter.

Seeing What the Machine Saw

A parser that writes 31 kilobytes of JSON about a one-page invoice needs a way for people to look at the result. The `html` visualizer turns a `vim.json` into a single HTML page with tabs for the structural, physical, semantic and table of contents layers, colour-coded blocks and a switch that hides auxiliary blocks such as page numbers. It runs on a finished file, so it works for any producer of VIM, not only for this parser:

```
portunix vim-parser visualize faktura-2026-0042.pdf.vim.json
```

For the people who maintain the parser, there is a second kind of sight: since March 2026, the parser can send OpenTelemetry traces, so that the time of each plugin on each document can be followed in a tracing tool.

The Same Model in Python

The second chapter told why the parser was ported to Python. Here is what the port looks like from the inside. The sub-package `ptx_reco.vim` in `portunix-reco` mirrors the Java modules: an `api` package with the plugin interfaces and the plugin context, a `core` package with the execution engine, the orchestration and a plugin registry, `extractors` for barcodes, full text, graphics, the lemmatizer, tables and the table of contents, and `parsers` with SPAYD. The model classes, nested classes in Java, became module-level Python dataclasses.

The library map of the translation decides what replaces what: PDFBox, ZXing, Tika and Lucene give way to their Python counterparts, and Jackson to dataclasses with explicit JSON serialization. The port also has something the Java parser does not: an OCR extractor that turns the image of a scanned page into physical blocks with provenance, because reco reads scans.

The port is the reason why VIM has two implementations today, and why every new layer must come with a plan for both, as the next chapter explains. The Python side writes the `extraction` and `prescriptive` layers already; it does not yet read them back, and the reco pipeline does not yet fill the extraction layer. The contract names reco as the producer of that layer; the documentation of the port says honestly that this is the planned state, not the current one.

A Consumer in Production: Invoices to ISDOC

The clearest example of VIM as a shared model is `portunix-ireco`, a pipeline for semi-structured documents such as invoices. Its first step, `parse`, runs the execution engine of `ptx_reco.vim` with the text block extractor and, where possible, the barcode, table, graphics, table of contents and metadata extractors. The result is a `vim.json` with the physical and structural layers. The second

step, `extract`, reads that file, finds the fields of the invoice with flexible layout templates and writes them into the semantic layer of the same file. A third step can export the result to ISDOC, the Czech XML standard for electronic invoices.

```
ptx-ireco parse invoice.pdf -o invoice.vim.json     # PDF → VIM (physical + structural)
ptx-ireco extract invoice.vim.json                  # fills the semantic layer in place
```

Two tools, two steps, one file. Each step adds its layers and leaves the others alone. That is what the sheets of glass of the first chapter were for.

Sources

- `plugins/vim-parser` in `portunix-plugins`: module READMEs, `Main.java`, the output of `auto --dry-run` and `plugins --deps` of version 0.1.8 (25 October 2026)
- `src/ptx_reco/vim/README.md` in `portunix-reco`
- `README.md` of `portunix-ireco`
- ADR-007 *Port vim-parser to Python via Dream Translation Tool*, `portunix-architecture`

CHAPTER 6

A Promise in JSON Schema

Two Implementations, One Truth

Once VIM had two implementations, a Java parser and a Python port, a quiet danger appeared. Two teams can read the same specification and understand it a little differently. A field written as `rawValue` on one side and `raw_value` on the other; a number that one side writes always and the other side only when it is not empty; a version string that one side checks and the other side ignores. Each difference is small. Together they make a file that one program writes and the other cannot read.

The answer of the ecosystem is a **contract**: a description of the format that is not code in any language, that a machine can check, and that lives in one place. For VIM that place is this repository, the directory `contract/schemas/`. ADR-004 decided it in March 2026. In October 2026 both implementations aligned their models with it, and the Java parser got a test that compares its model of the prescriptive layer with the schemas.

What the Contract Holds

The contract of VIM is a set of JSON Schema files in the Draft 2020-12 dialect:

Schema	What it describes
`vim-document.schema.json`	The root: `vim_version`, `document`, `layers`
`vim-extraction.schema.json`	The extraction layer: template reference, sources, merged document
`vim-extraction-source.schema.json`	One source of an extraction with its provenance
`vim-extraction-field.schema.json`	One extracted field with its value, raw value and revisions
`vim-prescriptive.schema.json`	The prescriptive layer: definition, references, terms
`vim-prescriptive-provision.schema.json`	One provision with anchors, relations and verification

Three rules of the root schema carry most of the weight.

The `layers` object is closed. It knows exactly six keys, from `physical` to `prescriptive`, and rejects any other. A producer cannot invent a layer of its own; a new layer is a change of the contract, decided in the open.

`vim_version` is a list of known values. It accepts the versions from `"1.0"` to `"2.1"` and nothing else.

Layers are bound to versions. A document with the `prescriptive` layer must say `"2.1"`. A document with the `extraction` layer must say `"2.0"` or `"2.1"`. This rule came on 16 October 2026 and was marked as a breaking change of the contract: documents that had a new layer with an old version number validated before and are rejected now. It ended the 1.x habit of the version as a list of features. In 2.x, the version is a promise about which layers a reader may find.

Examples That Must Pass and Examples That Must Fail

A schema that is never tested is a wish. The contract therefore comes with examples, and the rules of this repository say that every schema needs at least one. For VIM there are two valid examples, both already met in this book: `form-extraction.vim.json` with one scanned form and `definition-file.vim.json` with a definition file and its provisions.

Next to them, the directory `contract/examples/invalid/` holds forty VIM documents that must **fail**. Each one breaks exactly one rule: an anchor on page zero, a confidence above one, a provision without an ID, a reference with an unknown scope, a layer key that does not exist, an extraction layer with the version `"1.5"`. When someone loosens a rule by mistake, one of these documents suddenly passes, and the check fails.

The check is one command:

```
make validate-contract
```

It finds every example, picks the schema by the suffix of the file name (a `.vim.json` goes to `vim-document.schema.json`), and reports each result. A few lines of a real run:

```
PASS  [vim-document.schema.json] contract/examples/definition-file.vim.json
PASS  [vim-document.schema.json] contract/examples/form-extraction.vim.json
PASS  [vim-document.schema.json] contract/examples/invalid/anchor-page-zero.vim.json
(invalid as expected: /layers/prescriptive/provisions/0/anchors/0/page must be >= 1)
PASS  [vim-document.schema.json] contract/examples/invalid/prescriptive-wrong-vim-
version.vim.json (invalid as expected: /vim_version must be equal to constant)
...
98 checked, 0 failed, 8 skipped (no matching contract suffix).
```

What JSON Schema Cannot Say

Some rules are beyond JSON Schema. It can say that a provision has an ID; it cannot say that no two provisions have the same ID. It can say that a provision lists references; it cannot say that each listed reference exists in the same layer.

For these rules, the repository has a small extra check, `scripts/vim-identifier-check.mjs`, which runs after the schema check on every VIM example. It verifies that `provision_id`, `reference_id` and `term_id` are unique in the layer, and that every reference, term and relation of a provision points to something that exists. Its function `checkVimIdentifiers()` is exported, so other projects can import it and apply the same rules to their own documents. In the run above, the example `provision-reference-without-target.vim.json` fails here with the message *no reference with reference_id 'r-9'*.

When Is Something a New Layer?

The closed `layers` object raises a question that every format meets sooner or later: when new information arrives, where does it go? Barcodes became a collection inside the physical layer. The table of contents became a layer of its own. Each decision was made alone, and ADR-009 noticed that a wrong decision is expensive in both directions. An unnecessary layer needs a schema here, a section in the specification, a model in Java, a model in Python, support in the visualizer and

golden test files: the Java model of the extraction layer alone has 25 classes, and every layer is paid for twice. A missing layer, on the other hand, presses a foreign model into a layer with another meaning.

The answer is a **decision ladder**. New information takes the first step that fits:

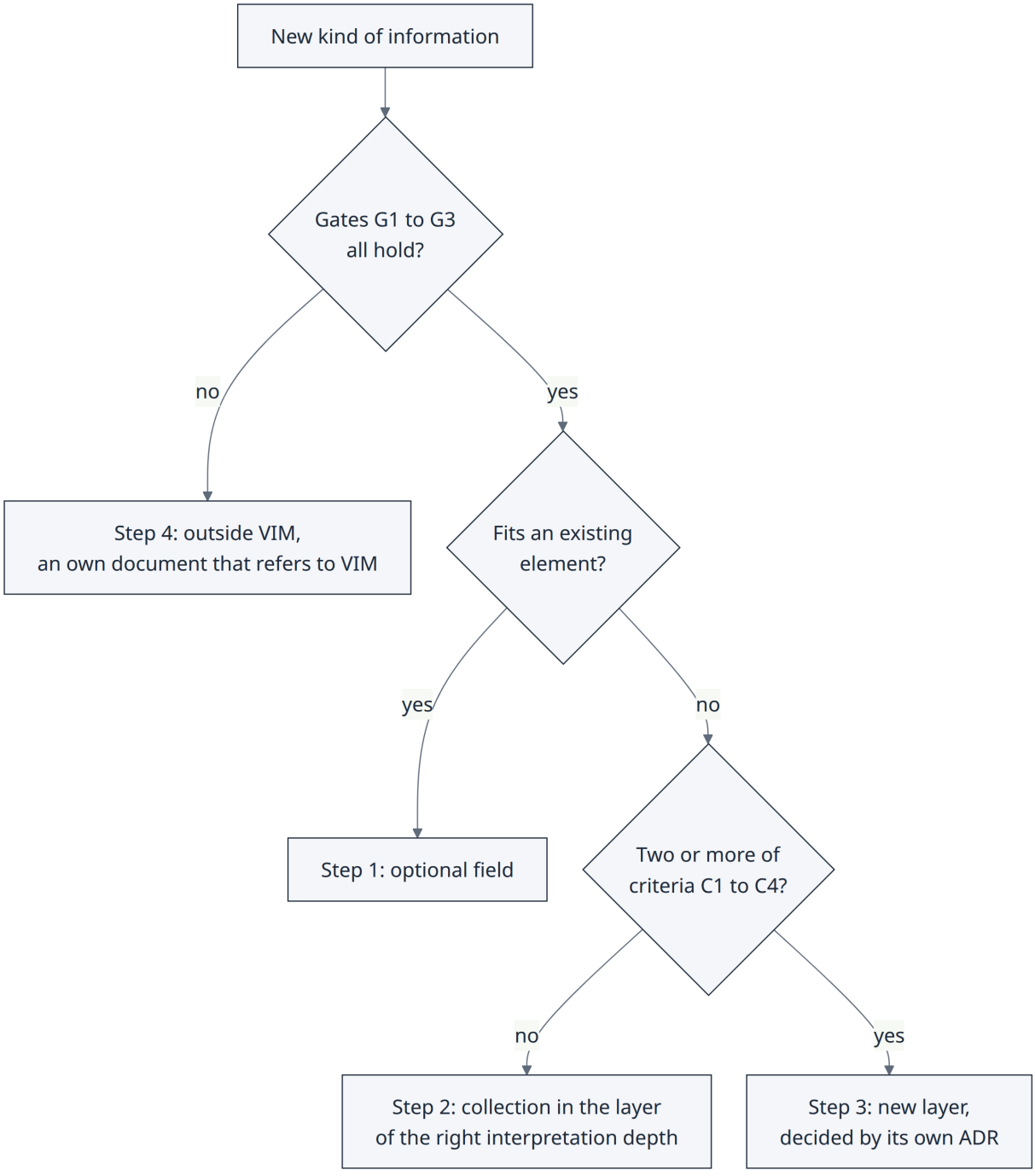


Figure 6.1 The decision ladder of ADR-009: a new VIM layer is the last step, after three gates and at least two of four criteria

The three **gates** decide whether the information belongs to VIM at all:

ID	Gate
G1	It is a result of understanding the file: derived from the document, meaningless without it

ID	Gate
G2	Every item can carry provenance: a source, a page, a region or the ID of another element
G3	It is useful to a consumer other than its producer

The four **criteria** decide whether it needs a layer of its own; at least two must be "yes":

ID	Criterion
C1	It has a different producer or life cycle than the output of vim-parser
C2	It will grow into its own domain model with several entity types and relations
C3	It will have revisions or state: values change after production
C4	It anchors into more than one layer or kind of element

Two small rules keep the count honest. When revisions are the only reason for a different life cycle, C1 and C3 count as one "yes". And the criteria are answered for the next twelve months, not for every possible future: "maybe later" is a "no".

The ladder was tested against history, and the result is instructive. The `extraction` layer scores three "yes" and would be a layer again. The SPAYD payments score none and are rightly a collection. And the `toc` layer scores none: under these rules it would be a collection in the structural layer. It stays a layer, because moving it would break every existing reader for no gain. The ADR says so plainly and adds that `toc` is not a precedent.

The first candidate to climb the whole ladder was the `prescriptive` layer, which answered "yes" to all three gates and all four criteria. Its ADR also shows what every new layer must bring: it is optional, it refers to other layers by ID instead of copying them, its key is one lower-case English noun, its schema comes **before** any code, and it has a plan for both implementations.

A Contract Is a Promise to Strangers

Other projects depend on the contracts of this repository, and many of their authors will never meet the authors of VIM. That is why the rules here are strict about change: every schema has a version of its own (`$version`), every breaking change is written into the changelog in bold, and every schema change comes with examples that pass and examples that fail. A contract is a promise to strangers. JSON Schema is only the language in which it is written.

The next part of the book leaves the inside of VIM and shows how to use it: how to make a `vim.json`, look at it, check it and read it from your own code.

Sources

- contract/README.md and contract/schemas/README.md of this repository
- scripts/vim-identifier-check.mjs and the output of `make validate-contract` (25 October 2026)
- CHANGELOG.md: #42 and #51
- ADR-004 and ADR-009, `portunix-architecture`

PART 4

Working with VIM

From a PDF to a checked vim.json in a few commands, and the open questions that shape the next versions.

7 · Your First vim.json

8 · Where VIM Is Going

CHAPTER 7

Your First vim.json

From Reading to Doing

Five chapters told the story of VIM and looked inside it. This one is for your hands. By its end, you will have made a `vim.json` from a PDF, looked at it in a browser, asked it two questions from a short Python program, and checked it against the contract. Every command and every output in this chapter was run on the sample invoice of the book, which is stored next to the chapters in `samples/`.

You need Java 21 for the parser, Python 3.11 or newer for the examples, and optionally `jq`, the small command line tool for JSON.

Step 1: Ask for the Plan

The parser is distributed as a directory with one JAR file and a `plugins/` directory of ZIP files. Inside Portunix, the same parser runs as `portunix vim-parser`; outside, you call the JAR directly. Before you run anything, ask the parser what it would do:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto --dry-run faktura-2026-0042.pdf
```

The answer is the execution plan you saw in the chapter about the machine that reads: seven plugins in three phases, and the lemmatizer left out because it is not enabled by default. The plan is a cheap way to see which plugins the parser found. If a plugin is missing from the list, its ZIP file is missing from `plugins/`.

Step 2: Make the File

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto -o faktura-2026-0042.pdf.vim.json faktura-2026
-0042.pdf
```

With `-o` the result goes to the given file; without it, to the standard output. The option `-w` writes it next to the PDF as a *sidecar* file with the suffix `.vim.json`. The most useful other options of the `auto` command are:

Option	Default	Effect
`--metadata` / `--no-metadata`	on	Document metadata with Schema.org names
`--fulltext`	off	The full text of the document in the physical layer
`--hash`	off	The SHA-256 hash of the file in the metadata
`--stemming`	off	Word stems in the semantic layer, for search
`--backend <id>`, `--lang <code>`	lucene, auto	Backend and language of stemming
`--exclude <plugins>`	none	Plugins to leave out, separated by commas

Option	Default	Effect
`--compact`	off	JSON without pretty printing

Step 3: Look at It

A `vim.json` of one page has hundreds of lines. Before you read them, look at the picture:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar visualize faktura-2026-0042.pdf.vim.json
```

The visualizer prints a single HTML page with a tab for each layer; with `-o` or `-w` it writes the page to a file. Open it in a browser, switch to the structural tab and find the block with the header *Odběratel*. Then switch to the physical tab and find the same block by its ID. You are now doing by hand what every consumer of VIM does in code: following an ID from a higher layer down to the evidence.

Step 4: Ask It Questions with jq

For a quick look from the command line, `jq` is enough. The headers of all structural blocks:

```
jq -r '.layers.structural.blocks[] | "\(.id)\t\(.header)"' faktura-2026-0042.pdf.vim.json
```

```
block_1 Faktura 2026-0042
block_2 Položka
block_3 null
block_4 Odběratel
block_5 MnožstvíCenaCelkem
```

The output shows the strengths and the weaknesses of the third chapter at a glance: the customer block has its header, the total has none, and the table header melted into one word.

Step 5: Ask It Questions from Code

A `vim.json` is plain JSON, so no special library is needed to read it. This short program answers the question of the prologue, *who is the customer*, and adds the evidence: the page and the position of the block the answer came from. Then it reads the payment:

```
import json
from pathlib import Path

doc = json.loads(Path("faktura-2026-0042.pdf.vim.json").read_text(encoding="utf-8"))
layers = doc["layers"]

# Who is the customer, and where on the page does it come from?
for entity in layers.get("semantic", {}).get("entities", []):
    if entity["type"] == "customer":
        block = next(b for b in layers["physical"]["blocks"] if b["id"] == entity["source_b
lock"])
        print(entity["fields"]["name"], "| page", block["page"], "| bbox", block["bbox"]["x
"], block["bbox"]["y"],
              "| confidence", entity["confidence"])

# What is there to pay?
for payment in layers.get("semantic", {}).get("payments", []):
    f = payment["fields"]
    print(f["amount"], f["currency"], "to", f["account"], "VS", f["variable_symbol"])
```

```
Jana Příkladová | page 1 | bbox 299.13782 155.6819 | confidence 0.8
1250.0 CZK to CZ6508000000192000145399 VS 20260042
```

Notice the pattern, because it is the pattern of every good VIM consumer. It reads the meaning from the semantic layer, follows `source_block` down to the physical layer for the evidence, and keeps the confidence next to the answer. A program that drops the confidence on the way turns an educated guess into a false fact.

All layers are optional, so the program uses `.get()` with an empty default for every layer. A `vim.json` from form recognition may have no semantic layer at all.

Step 6: Check It against the Contract

The contract of the fifth chapter can check your own files too, not only the examples of this repository. Any validator of JSON Schema Draft 2020-12 will do, as long as it loads all `vim-*.schema.json` files, because the root schema refers to the others by their `$id`. In Python, with the libraries `jsonschema` and `referencing`:

```
import json
import sys
from pathlib import Path

from jsonschema import Draft202012Validator
from referencing import Registry, Resource

schemas = Path(sys.argv[1])
registry = Registry().with_resources(
    (s["$id"], Resource.from_contents(s))
    for s in (json.loads(p.read_text(encoding="utf-8")) for p in schemas.glob("vim-
*.schema.json"))
)
root = json.loads((schemas / "vim-document.schema.json").read_text(encoding="utf-8"))
validator = Draft202012Validator(root, registry=registry)
doc = json.loads(Path(sys.argv[2]).read_text(encoding="utf-8"))
errors = sorted(validator.iter_errors(doc), key=lambda e: list(e.path))
for e in errors:
    print("/" + "/".join(map(str, e.path)), e.message)
print("valid" if not errors else f"{len(errors)} error(s)")
```

For the sample invoice it prints `valid`. For one of the negative examples of the contract, an extraction layer with a version that is too old, it prints:

```
/vim_version '1.5' is not one of ['2.0', '2.1']
1 error(s)
```

If your document has a prescriptive layer, run the identifier check after the schema check; `checkVimIdentifiers()` in `scripts/vim-identifier-check.mjs` finds duplicate IDs and links to provisions, references and terms that do not exist.

Step 7: Two Tools, One File

The last step shows VIM as a shared container. The invoice pipeline of `portunix-ireco` works in two commands on one file:

```
ptx-ireco parse invoice.pdf -o invoice.vim.json     # PDF → VIM (physical + structural)
ptx-ireco extract invoice.vim.json                  # fills the semantic layer in place
```

The first command writes the physical and structural layers with the Python port of the parser. The second one reads them and adds the semantic layer, without touching what is already there. Your own tool can join such a chain in the same way: read the layers you need, write the layer you own, and leave the rest alone.

Try It Yourself

1. Run the parser on the sample invoice with `--exclude barcode`. What happens, and what must you add to the option to get a `vim.json` without barcodes?
2. Change the Python program so that it prints the supplier. What does it print, and what does that tell you about the semantic layer of this sample?
3. Write a `vim.json` by hand with only the `extraction` layer and `vim_version "1.5"`. What does the validator say? What must you change?
4. Find a sentence in this chapter that explains why a consumer should keep the confidence. Then find a value in the sample whose confidence is too high for what it says.

Answers

1. The parser stops before it reads anything: *Plugin 'spayd' depends on missing plugin 'barcode'*. The orchestrator does not quietly drop the plugins that depend on an excluded one; it asks you to decide. The graphics extractor depends on the barcodes too, so the working option is `--exclude barcode,spayd,graphics`. Then the physical layer has empty `barcodes` and `graphics`, and the semantic layer has empty `payments`. Check such a change with `--dry-run` first.
2. There is no entity of type `supplier`. The supplier's address landed in `block_1` together with the title, and the entity made from that block has the type `invoice` and the name *Dodavatel*. The semantic layer is a heuristic, and on this layout it fails.
3. `/vim_version '1.5' is not one of ['2.0', '2.1']`. The extraction layer exists since version 2.0, so the document must say `"2.0"` or `"2.1"`.

4. The entity of type `invoice` from `block_1` has the confidence 0.8, the same as the correct customer, although its `city` is a date. The confidence of a heuristic says how sure the heuristic is, not how right it is.

The last chapter looks ahead: what VIM still cannot do well, and where it is going.

CHAPTER 8

Where VIM Is Going

The Honest Middle

Every young format lives in an honest middle. It is good enough to be used every day, and not yet good enough to be forgotten. VIM is there now. The sample invoice of this book showed both halves on one page: the customer and the payment read right, with their evidence, and a title that swallowed the supplier, a table whose columns melted. The format held both kinds of result equally well. The reading itself still has a long way to go.

This last chapter looks at that way: the problems that are known, the place where they are studied, the questions that the architecture decisions left open on purpose, and the idea that holds it all together.

A Laboratory for Hard Pages

In May 2026, the team founded a separate repository for the research on the parser, `portunix-vim-parser-research-lab`. It has an unusual rule: it never changes the parser directly. It keeps the parser as a read-only submodule, runs it against collections of documents, compares it with other tools and hands its findings to the parser team as issues.

The lab works with two kinds of records. A **catalog entry** describes a known extraction problem and lives for a long time; it is not closed just because one experiment ended. A **challenge** is one measurement: a versioned, repeatable comparison with a clear win condition and a written verdict. The first three catalog entries are exactly the problems that any reader of PDFs meets sooner or later:

ID	Problem	What goes wrong
I-PDF-001	Page footers	Page numbers, running titles and copyright lines appear in the text at every page break
I-PDF-002	Multi-column layout	Text read in stored order zig-zags between the columns
I-PDF-003	Tables across page breaks	One logical table becomes two, with the header row twice

The first challenge, C001, compares `vim-parser` with Marker, an open-source converter from PDF to Markdown, on these three problems. At the time of writing it is a stub: the pipeline is defined, the runs have not started. That is not a weakness of the lab. It is its method: first agree on what will be measured and what counts as a win, then measure.

The melted columns of our invoice belong to the family of I-PDF-002, and the table that the detector found over a large part of the page belongs to the family of I-PDF-003. The sample of this book could become a small entry in that collection.

Questions Left Open on Purpose

Good architecture decisions do not pretend to know everything. The ADRs of VIM end with open questions, and they are worth listing, because they show where the format will grow.

A place for experiments. The `layers` object is closed, and a new layer needs a schema and two implementations. That protects consumers, but it leaves no place for content that is not yet stable enough for a contract. ADR-009 names a possible *extension area* for such content and leaves it as a separate decision.

A registry of layers. With six layers, the list of layers with their owners and producers lives comfortably in the format specification. With more, a short registry may be needed.

A language for rules. A provision of the prescriptive layer can say that it is checked automatically, and it carries a `rule` object. What may be written in that object is not fixed by the contract; tools must agree on it outside. A small rule language is an open question of ADR-010, together with the format of the assessment document that records the result of a compliance check.

The home of the LLM producer. The prescriptive layer is written by a producer that uses a language model. Where that producer lives, and how it is run, is also still open; ADR-010 suggests that tools which use a language model at development time run as Claude Code skills.

The second half of the Python port. The Python model writes the extraction and prescriptive layers but does not read them back, and the recognition pipeline of reco does not fill the extraction layer yet. Until it does, the contract names a producer that is planned rather than real, and the documentation says so.

Small Truths for Large Models

There is a temptation in the age of large language models to think that structure no longer matters. A model can read a PDF, or even a photograph of a page, and answer questions about it. Why keep three sheets of glass, IDs and confidences, when a model can simply look?

The experiment of the prologue is the first answer. The same model, given the same document, was unsure with plain text and sure with the layers. Structure is not a crutch for weak readers; it is how a strong reader shows its work. The second answer is provenance. A model that answers *Jana Příkladová* is useful. A system that answers *Jana Příkladová, page 1, block 4, confidence 0.8, found by matching the label Odběratel* can be checked, corrected and trusted. When we look closer at the reading of documents, we find a varied world of small, patient skills: grouping, labelling, measuring, doubting. It is easy to overlook them when we chase new goals. VIM keeps them, and it keeps them in a form that a person, a program and a language model can all read.

The prescriptive layer points to the third answer. There, a language model is invited in as a producer, and the format makes room for it honestly: every provision it proposes starts as `proposed`, and only a person makes it `confirmed`. The model brings speed; the layers bring evidence; the person brings the final word.

Back to the Glass

We started with three sheets of glass over a page. Nine months later there are six, and more may come, each one through the gates and criteria of the ladder. The idea underneath has not changed: keep what is there, keep how it is grouped, keep what it means, and let every value say where it came from and how sure we are.

A document that went through VIM can say what it means. That was the promise of the name, and it is the promise that the next versions will keep working on.

Sources

- `README.md`, `docs/catalog/pdf/` and `challenges/C001-pdf-vs-marker/` of `portunix-vim-parser-research-lab`
- ADR-009 and ADR-010, section *Open Questions*, `portunix-architecture`
- `src/ptx_reco/vim/README.md` of `portunix-reco`